

11주차 · 11/12(목)

JavaScript 코어 객체 활용

Array, String, Math 등 주요 객체 다루기 · 함수와 객체

한신대학교 AI·SW계열 · 웹프로그래밍
학습사이트 hs-web.dreamitbiz.com

학습 목표

- ① 함수를 만들어 반복되는 코드를 묶고 재사용할 수 있다.
- ② 화살표 함수를 읽고 쓸 수 있다.
- ③ Array 메서드(map·filter·find·reduce)를 상황에 맞게 골라 쓸 수 있다.
- ④ String·Math 등 내장 객체의 주요 메서드를 활용할 수 있다.
- ⑤ 객체로 여러 값을 하나로 묶고, 배열과 조합해 자료를 다룰 수 있다.
- ⑥ 값이 복사되는 것과 참조가 복사되는 것의 차이를 설명할 수 있다.

11주차

핵심 키워드

function

화살표 함수

return

스코프

Array

map/filter/find/reduce

String

Math

객체

구조 분해

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

11주차 · 1

함수 — 코드에 이름 붙이기

같은 코드를 세 군데에 복사해 붙였다면, 고칠 일이 생겼을 때 세 군데를 다 고쳐야 합니다.

함수 — 코드에 이름 붙이기

- 1 하나라도 빠뜨리면 버그입니다.
- 2 함수는 그 코드에 이름을 붙여 한 곳에만 두는 것입니다.

함수 만들고 쓰기

```
1 // 선언
2 function add(a, b) {      // a, b = 매개변수(parameter)
3     return a + b;        // return = 결과를 돌려준다
4 }
5
6 // 호출
7 const result = add(3, 4);  // 3, 4 = 인자(argument)
8 console.log(result);      // 7
9
10 // return이 없으면 undefined를 돌려줍니다
11 function greet(name) {
12     console.log(`안녕, ${name}`); // 출력만 하고 돌려주지는 않음
13 }
14 const r = greet('한신');    // r은 undefined
```

함수 — 코드에 이름 붙이기

- 1 `console.log()` 로 출력하는 것과 `return` 으로 돌려주는 것은 완전히 다릅니다.
- 2 출력은 사람이 보는 것이고, `return`은 다른 코드가 이어서 쓰는 것입니다.
- 3 초보자가 가장 자주 헷갈리는 지점입니다.

요즘 더 많이 쓰는 모양 (1/2)

```
1 // 함수 선언
2 function add(a, b) { return a + b; }
3
4 // 함수 표현식
5 const add2 = function (a, b) { return a + b; };
6
7 // 화살표 함수
8 const add3 = (a, b) => { return a + b; };
9
10 // 몸통이 return 하나면 중괄호와 return을 생략할 수 있습니다
```

요즘 더 많이 쓰는 모양 (2/2)

```
19 const add4 = (a, b) => a + b;
20
21 // 매개변수가 하나면 괄호도 생략 가능 (팀 관례에 따라 붙이기도 합니다)
22 const double = n => n * 2;
23
24 // 매개변수가 없으면 빈 괄호
25 const now = () => new Date();
26
27 // 객체를 돌려줄 때는 괄호로 감쌉니다 — 안 그러면 중괄호를 몸통으로 봅니다
28 const makeUser = (name) => ({ name, year: 1 });
```

값이 안 넘어왔을 때

```
1 function greet(name = '손님', greeting = '안녕하세요') {  
2   return `${greeting}, ${name}님!`;  
3 }  
4  
5 greet();           // '안녕하세요, 손님님!'  
6 greet('한신');     // '안녕하세요, 한신님!'  
7 greet('한신', '반가워요'); // '반가워요, 한신님!'
```

중괄호 밖에서는 안 보입니다

```
1  const global = '전역';
2
3  function outer() {
4      const local = '함수 안';
5      console.log(global); // ✅ 밖의 것은 안에서 보인다
6  }
7
8  console.log(local);      // ❌ ReferenceError: local is not defined
9
10 if (true) {
11     const blockVar = '블록 안';
12 }
13 console.log(blockVar);   // ❌ const/let은 블록 밖에서 안 보인다
14
15 // ⚠ var는 블록을 무시하고 새어 나옵니다. 그래서 쓰지 않습니다.
```

좋은 함수의 조건

- 1 이름이 하는 일을 말한다**
calc() 가 아니라 calculateTotalPrice().
- 2 한 가지 일만 한다**
이름에 "그리고"가 들어가면 두 개로 나눌 때입니다.
- 3 밖의 것을 몰래 바꾸지 않는다**
받은 값으로 계산해 돌려주는 함수가 가장 다루기 쉽습니다.
- 4 짧다**
화면을 넘어가면 나누는 것을 고려하세요.

11주차 · 2

배열 — 순서 있는 값 묶음

기본

```
1  const fruits = ['사과', '배', '감'];
2
3  fruits[0]      // '사과'   ← 0부터 셉니다
4  fruits[2]      // '감'
5  fruits[3]      // undefined (없는 자리)
6  fruits.length  // 3
7  fruits[fruits.length - 1] // 마지막 요소
8  fruits.at(-1)   // '감'   ← 뒤에서 세기 (더 편합니다)
```

추가·제거

```
1  const a = [1, 2, 3];
2
3  a.push(4);      // 뒤에 추가 → [1,2,3,4]
4  a.pop();        // 뒤에서 빼기 → 4를 돌려주고 [1,2,3]
5  a.unshift(0);   // 앞에 추가 → [0,1,2,3]
6  a.shift();      // 앞에서 빼기 → 0을 돌려주고 [1,2,3]
7
8  a.splice(1, 1);  // 1번 자리부터 1개 삭제 → [1,3]
9  a.splice(1, 0, 99); // 1번 자리에 99 끼워넣기 → [1,99,3]
10
11 // △ 위 메서드들은 원본을 바꿉니다.
```

2.1 배열 메서드 — 여기가 핵심입니다

- 1 실무에서 배열을 다룰 때는 for 문보다 아래 메서드를 훨씬 많이 씁니다.
- 2 무엇을 하려는지가 코드에 그대로 드러나기 때문입니다.

2.1 배열 메서드 — 여기가 핵심입니다 (1/2)

메서드	하는 일	돌려주는 것
forEach	하나씩 실행	없음(undefined)
map	각각을 변형	같은 길이의 새 배열
filter	조건에 맞는 것만 고르기	짧아진 새 배열
find	조건에 맞는 첫 하나	요소 하나 (없으면 undefined)

2.1 배열 메서드 — 여기가 핵심입니다 (2/2)

메서드	하는 일	돌려주는 것
some	하나라도 맞는가	true / false
every	전부 맞는가	true / false
reduce	전부 합쳐 하나로	결과값 하나
sort	정렬	정렬된 배열 (원본도 바뀜)

map — 각각을 바꿔 새 배열

```
1 const nums = [1, 2, 3];
2 const doubled = nums.map(n => n * 2);           // [2, 4, 6]
3 console.log(nums);                             // [1, 2, 3] ← 원본 그대로
4
5 const names = ['한신', '길동'];
6 const tags = names.map(n => `<li>${n}</li>`);    // HTML 만들기에 자주 씁니다
```

filter — 조건에 맞는 것만

```
1  const scores = [95, 42, 88, 67];
2  const passed = scores.filter(s => s >= 60);    // [95, 88, 67]
3
4  const students = [
5    { name: '한신', year: 1 },
6    { name: '길동', year: 2 },
7    { name: '영희', year: 1 },
8  ];
9  const freshmen = students.filter(s => s.year === 1);  // 1학년만
```

find — 첫 하나만

```
1  const found = students.find(s => s.name === '길동');  
2  // { name: '길동', year: 2 }  
3  
4  // filter는 배열을, find는 요소 하나를 돌려줍니다. 자주 헷갈립니다.  
5  students.filter(s => s.name === '길동')    // [{ ... }] ← 배열  
6  students.find(s => s.name === '길동')      // { ... }   ← 객체
```

reduce — 하나로 합치기

```
1  const nums = [10, 20, 30];
2
3  const sum = nums.reduce((acc, n) => acc + n, 0);
4  //           ↑      ↑      ↑
5  //           누적값 현재값 시작값
6  // 진행: (0,10)→10 · (10,20)→30 · (30,30)→60
7  console.log(sum); // 60
8
9  const max = nums.reduce((a, b) => (a > b ? a : b));
10 const total = students.reduce((acc, s) => acc + s.year, 0);
```

sort — ⚠ 함정이 있습니다

```
1  const nums = [10, 9, 100, 2];
2
3  nums.sort();           // [10, 100, 2, 9] ← 문자열로 정렬해서 엉망!
4  nums.sort((a, b) => a - b); // [2, 9, 10, 100] ← 오름차순 ✓
5  nums.sort((a, b) => b - a); // [100, 10, 9, 2] ← 내림차순
6
7  // 한국어 이름 정렬
8  names.sort((a, b) => a.localeCompare(b));
9
10 // ⚠ sort는 원본을 바꿉니다. 원본을 두려면 복사 후 정렬합니다.
11 const sorted = [...nums].sort((a, b) => a - b);
```

메서드 이어 쓰기 (체이닝)

```
1  const result = students
2    .filter(s => s.year === 1)      // 1학년만 고르고
3    .map(s => s.name)                // 이름만 뽑아
4    .sort((a, b) => a.localeCompare(b)) // 정렬
5    .join(', ');                    // 문자열로 합치기
6
7  console.log(result);    // '영희, 한신'
```

11주차 · 3

객체 — 이름표가 붙은 값 묶음

만들고 꺼내 쓰기

```
1  const student = {
2    name: '김한신',
3    year: 1,
4    dept: 'AI · SW계열',
5    subjects: ['웹프로그래밍', '자료구조'],
6    greet: function () {           // 값이 함수여도 됩니다 (메서드)
7      return `안녕, 나는 ${this.name}`;
8    },
9  };
10
11  student.name           // '김한신'   ← 점 표기 (기본)
12  student['name']        // '김한신'   ← 대괄호 표기
13  const key = 'year';
14  student[key]           // 1   ← 변수로 접근할 때는 대괄호만 가능
15
16  student.greet()        // '안녕, 나는 김한신'
17  student.grade          // undefined (없는 키)
```

추가·수정·삭제·확인

```
1 student.email = 'me@example.com'; // 추가
2 student.year = 2; // 수정
3 delete student.dept; // 삭제
4
5 'name' in student // true
6 Object.keys(student) // ['name', 'year', ...]
7 Object.values(student) // 값들의 배열
8 Object.entries(student) // [['name', '김한신'], ...]
```

자주 보게 될 문법입니다

```
1  const { name, year } = student;
2  console.log(name, year);      // '김한신' 1
3
4  // 이름을 바꿔 받기
5  const { name: studentName } = student;
6
7  // 없을 때 기본값
8  const { grade = 'N/A' } = student;
9
10 // 배열도 됩니다
11 const [first, second] = ['사과', '배'];
12
13 // 함수 매개변수에서 바로 분해 (실무에서 아주 흔합니다)
14 function printStudent({ name, year }) {
15   console.log(`${name} (${year}학년)`);
16 }
17 printStudent(student);
```

... 세 점

```
1  const a = [1, 2];
2  const b = [...a, 3];           // [1, 2, 3] ← a는 그대로
3
4  const base = { name: '한신', year: 1 };
5  const updated = { ...base, year: 2 }; // { name: '한신', year: 2 }
6
7  // 두 배열 합치기
8  const merged = [...arr1, ...arr2];
9
10 // 함수 인자로 펼치기
11 Math.max(...[3, 1, 4]); // 4
```

가장 헷갈리는 지점입니다 (1/2)

```
1 // 기본형 — 값이 복사됩니다
2 let a = 1;
3 let b = a;
4 b = 2;
5 console.log(a); // 1 ← a는 그대로
6
7 // 참조형 — 주소가 복사됩니다 (같은 것을 가리킴)
8 const x = { n: 1 };
9 const y = x;
10 y.n = 2;
```

가장 헷갈리는 지점입니다 (2/2)

```
19 console.log(x.n); // 2 🤖 x도 바뀝니다!
20
21 // 진짜 복사하려면 펼침을 씁니다
22 const z = { ...x };
23 z.n = 3;
24 console.log(x.n); // 2 ← 이번엔 안 바뀝니다
25
26 // ⚠ 펼침은 "얕은 복사"입니다. 안에 또 객체가 있으면 그건 여전히 공유됩니다.
27 const deep = structuredClone(x); // 깊은 복사가 필요할 때
```

11주차 · 4

배열 + 객체 — 실전 자료 구조

거의 모든 웹 데이터가 이 모양입니다 (1/2)

```
1  const products = [  
2    { id: 1, name: '노트북', price: 1200000, category: '전자', stock: 5 },  
3    { id: 2, name: '마우스', price: 25000, category: '전자', stock: 0 },  
4    { id: 3, name: '노트', price: 3000, category: '문구', stock: 42 },  
5  ];  
6  
7  // 재고 있는 전자제품만  
8  const available = products.filter(p => p.category === '전자' && p.stock > 0);  
9  
10 // 이름만 뽑기  
11 const names = products.map(p => p.name);  
12  
13 // 총 재고 금액
```

거의 모든 웹 데이터가 이 모양입니다 (2/2)

```
19 const totalValue = products.reduce((sum, p) => sum + p.price * p.stock, 0);
20
21 // id로 찾기
22 const item = products.find(p => p.id === 2);
23
24 // 가격순 정렬 (원본 보존)
25 const cheap = [...products].sort((a, b) => a.price - b.price);
26
27 // 화면에 그릴 HTML 만들기
28 const html = products
29   .map(p => `- ${p.name} — ${p.price.toLocaleString()}원</li>`)
30   .join('');

```

객체 ↔ 문자열

```
1  const obj = { name: '한신', year: 1 };
2
3  const text = JSON.stringify(obj);
4  // '{"name":"한신","year":1}' ← 문자열
5
6  const back = JSON.parse(text);
7  // { name: '한신', year: 1 } ← 객체로 복원
8
9  // 보기 좋게 (들여쓰기 2칸)
10 JSON.stringify(obj, null, 2);
11
12 // ⚠ JSON은 키를 반드시 큰따옴표로 감쌉니다. 함수와 undefined는 담기지 않습니다.
```

4.1 JSON — 자료를 주고받는 표준 형식

- 1 JSON은 12주차에 서버에서 자료를 받아 올 때, 그리고 `localStorage` 에 저장할 때 반드시 나옵니다.
- 2 `localStorage` 는 문자열만 저장하므로 `stringify` 와 `parse` 가 늘 짝으로 나옵니다.

11주차 · 5

String — 문자열 객체

자주 쓰는 문자열 메서드 (1/2)

```
1  const s = '  Hello Web Programming  ';  
2  
3  s.length           // 25  
4  s.trim()           // 앞뒤 공백 제거 — 입력값 검사에 필수  
5  s.toUpperCase()     // 대문자로  
6  s.toLowerCase()    // 소문자로  
7  s.includes('Web')   // true — 포함하는가  
8  s.startsWith('  He') // true  
9  s.endsWith('  ')    // true  
10 s.indexOf('Web')    // 8 — 없으면 -1
```

자주 쓰는 문자열 메서드 (2/2)

```
19 s.replace('Web', '웹')           // 첫 하나만 바꾼다
20 s.replaceAll(' ', '_')          // 전부 바꾼다
21 s.split(' ')                    // 공백으로 잘라 배열로
22 s.slice(2, 7)                   // 일부 잘라내기
23 s.padStart(3, '0')              // '007' 처럼 앞을 채우기
24 s.repeat(2)                     // 두 번 반복
25
26 // ⚠ 문자열은 바뀌지 않습니다(immutable).
27 //   s.toUpperCase()는 새 문자열을 돌려줄 뿐 s 자체는 그대로입니다.
```

11주차 · 6

Math — 수학 객체

Math는 new 없이 바로 씁니다

```
1  Math.round(3.6)      // 4   반올림
2  Math.floor(3.9)      // 3   내림
3  Math.ceil(3.1)       // 4   올림
4  Math.trunc(-3.9)     // -3   소수점 버리기(음수 주의: floor는 -4)
5  Math.abs(-5)         // 5   절댓값
6  Math.max(1, 5, 3)    // 5
7  Math.min(1, 5, 3)    // 1
8  Math.pow(2, 10)      // 1024 (2 ** 10 과 같음)
9  Math.sqrt(16)        // 4
10 Math.PI              // 3.141592653589793
11 Math.random()        // 0 이상 1 미만 난수
12
13 // 1~n 무작위 정수 — 자주 쓰는 관용구
14 function randInt(n) { return Math.floor(Math.random() * n) + 1; }
15
16 // 배열에서 무작위로 하나 고르기
17 const arr = ['가위', '바위', '보'];
18 arr[Math.floor(Math.random() * arr.length)]
```

Math — 수학 객체

- 1 **Math.max(...arr)** 처럼 펼침(spread) 과 함께 쓰면 배열의 최댓값을 구할 수 있습니다.
- 2 **Math.max(arr)** 는 배열을 숫자로 못 바꿔 NaN 이 나옵니다 — 자주 하는 실수입니다.

11주차 · 7

Number — 숫자 다루기

변환과 검사

```
1  Number('42')           // 42
2  Number('42px')          // NaN
3  parseInt('42px')        // 42    앞에서부터 숫자만
4  parseFloat('3.14m')     // 3.14
5
6  Number.isNaN(x)         // NaN인가 (전역 isNaN보다 정확)
7  Number.isInteger(4.0)   // true
8
9  (3.14159).toFixed(2)    // '3.14' ← 문자열로 돌려줍니다
10 (1234567).toLocaleString('ko-KR') // '1,234,567'
11
12 console.log(0.1 + 0.2); // 0.30000000000000004 ← 컴퓨터 소수 표현의 한계
13 (0.1 + 0.2).toFixed(2); // '0.30'
```

11주차

실습

이 시간의 실습 4개 — 학습사이트의 코드 실습장에서 직접 치고 실행합니다

실습 1 · 기본

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“원의 반지름을 받아 넓이를 돌려주는 함수 `circleArea`를 만들고, 반지름 5의 결과를 소수점 둘째 자리까지 출력하세요.

힌트

넓이는 $\text{Math.PI} * r * r$ 입니다. 소수점은 `toFixed(2)`로 자릅니다. 출력이 아니라 `return`으로 돌려주는 것이...

실습 1 모범답안

```
1 function circleArea(r) {  
2   return Math.PI * r * r;  
3 }  
4  
5 console.log(circleArea(5).toFixed(2)); // 78.54
```

실습 2 · 기본

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“점수 배열 [95, 42, 88, 67, 55]에서 60점 이상만 골라 새 배열을 만들고, 그 개수와 평균을 출력하세요.

힌트

filter로 고르고, **reduce**로 합을 구한 뒤 **length**로 나눕니다.

실습 2 모범답안

```
1  const scores = [95, 42, 88, 67, 55];
2
3  const passed = scores.filter(s => s >= 60);
4  const sum = passed.reduce((acc, s) => acc + s, 0);
5  const avg = sum / passed.length;
6
7  console.log('합격:', passed);
8  console.log('인원:', passed.length);
9  console.log('평균:', avg.toFixed(1));
```

실습 3 · 응용

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“학생 객체 배열을 만들고(이름·학년·점수 3명 이상), 1학년만 골라 이름만 뽑은 뒤 심표로 이어 출력하세요.

힌트

filter → **map** → **join** 을 이어서 씁니다. 체이닝이라고 부릅니다.

실습 3 모범답안

```
1  const students = [  
2    { name: '김한신', year: 1, score: 88 },  
3    { name: '이길동', year: 2, score: 74 },  
4    { name: '박영희', year: 1, score: 92 },  
5  ];  
6  
7  const freshmen = students  
8    .filter(s => s.year === 1)  
9    .map(s => s.name)  
10   .join(', ');  
11  
12  console.log('1학년:', freshmen);
```

실습 4 · 심화

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“상품 배열에서 ① 총 재고 금액 ② 재고가 0인 상품 이름 ③ 가장 비싼 상품을 구해 출력하세요. 상품은 id·name·price·stock을 가집니다.

힌트

reduce로 합계, filter로 품질, sort나 reduce로 최고가를 구합니다. 원본을 정렬하지 않도록 복사해서...

실습 4 모범답안

```
1  const products = [  
2    { id: 1, name: '노트북', price: 1200000, stock: 5 },  
3    { id: 2, name: '마우스', price: 25000, stock: 0 },  
4    { id: 3, name: '노트', price: 3000, stock: 42 },  
5  ];  
6  
7  // ① 총 재고 금액  
8  const totalValue = products.reduce((sum, p) => sum + p.price * p.stock, 0);  
9  console.log('총 재고 금액:', totalValue.toLocaleString() + '원');  
10  
11 // ② 품절 상품  
12 const soldOut = products.filter(p => p.stock === 0).map(p => p.name);  
13 console.log('품절:', soldOut);  
14  
15 // ③ 가장 비싼 상품 (원본을 건드리지 않으려고 복사 후 정렬)  
16 const priciest = [...products].sort((a, b) => b.price - a.price)[0];  
17 console.log('최고가:', priciest.name);
```

11주차 마무리

11주차 마무리

JavaScript 코어 객체 활용 — 이 시간에 다룬 것을 학습사이트에서 다시 확인합니다

✓ 학습사이트 주차 페이지의 절마다 「이해했어요 · 실습했어요 · 어려워요」 표시

✓ 퀴즈 9 「퀴즈 9 — JavaScript 코어 객체」 응시 (성적 미반영 · 학습 확인용)

✓ 실습 4개를 실습장에서 「확인」 까지 — 「실습했어요」 체크

✓ 막히면 「질문있어요」 또는 패들릿에 — 30분 넘게 혼자 막혀 있지 않기

다음 시간

12주차 · HTML5 DOM 활용